API - Manual de Funcionamento Endpoint por Endpoint

Documentacao tecnica - Manual Operacional Digital

Fundamentos

A API Express usa JSON, `multipart/form-data` e JWT. A URL local é `http://localhost:4000`. Rotas privadas exigem `Authorization: Bearer TOKEN`. O middleware `authenticate` valida assinatura e expiração e grava o payload em `req.user`.

Os perfis são `usuario`, `gerente_setor` e `suporte`. `requireManualManager` aceita gerente de setor e suporte; `requireSupport` aceita suporte. O código ainda reconhece `gerente` legado em alguns pontos.

1. `GET /health`

Acesso: público. Objetivo: confirmar que o processo HTTP está ativo. Não testa PostgreSQL nem Storage.

Fluxo: Express responde diretamente com `status: ok` e o nome do serviço.

2. `POST /api/auth/login`

Acesso: público. Entrada: `identificador` e `senha` em JSON.

Fluxo interno: valida campos; verifica bloqueio por IP+identificador; consulta usuário por identificador ou e-mail; compara o hash bcrypt; registra falhas; após cinco erros bloqueia por 15 minutos; em sucesso limpa as falhas, grava auditoria e assina JWT por oito horas.

Saída: token e usuário seguro, sem senha. Erros: 400, 401 e 429.

```
{"identificador":"admin","senha":"admin123"}
```

3. `GET /api/auth/me`

Acesso: autenticado. Fluxo: lê o ID do JWT, consulta usuário e setor e devolve a sessão atual. O frontend usa esta rota ao recarregar a página.

4. `PUT /api/auth/password`

Acesso: autenticado. Entrada: senha atual e nova senha.

Fluxo: exige oito caracteres, letra e número; compara a senha atual; gera novo bcrypt; atualiza `users.senha`; grava `password_changed`; responde 204.

5. `POST /api/auth/register`

Acesso: suporte. Entrada: nome, identificador, senha forte, perfil e setor.

Fluxo: valida perfil; cria ou reutiliza setor; gera bcrypt; insere usuário; devolve dados sem senha. Identificador duplicado gera erro do banco tratado pelo middleware geral.

6. GET /api/users

Acesso: suporte. Fluxo: consulta usuários com `LEFT JOIN sectors`, ordena por nome e omite senha. Usado na tela administrativa.

7. PUT /api/users/{id}

Acesso: suporte. Entrada: nome, identificador, perfil e setor, todos obrigatórios.

Fluxo: valida perfil; cria/reutiliza setor; atualiza identificação, perfil e setor; devolve o registro atualizado. Não altera senha.

8. DELETE /api/users/{id}

Acesso: suporte. Fluxo: impede excluir o próprio ID; remove usuário. Cascatas removem progresso, tentativas e permissões; autoria e auditoria podem manter o registro com usuário nulo. Responde 204.

9. GET /api/systems

Acesso: autenticado. Fluxo: lista todos os sistemas e calcula `manuals_count`. A contagem depende do setor do usuário, embora o catálogo de sistemas seja global.

10. POST /api/systems

Acesso: suporte. Entrada: `nome` obrigatório e `resumo` opcional. Fluxo: insere e devolve o sistema com status 201.

11. PUT /api/systems/{id}

Acesso: suporte. Entrada: nome e/ou resumo. Fluxo: usa `COALESCE` para manter campos ausentes, atualiza `updated_at` e devolve 404 se o ID não existir.

12. DELETE /api/systems/{id}

Acesso: suporte. Fluxo: exclui o sistema. A FK define `manuals.system_id` como nulo, preservando os manuais. Responde 204.

13. GET /api/sectors

Acesso: gerente de setor ou suporte. Fluxo: lista ID e nome em ordem alfabética. O frontend usa a resposta em cadastros.

14. POST /api/sectors

Acesso: suporte. Entrada: nome não vazio. Fluxo: `INSERT ... ON CONFLICT` cria ou reutiliza o setor e devolve 201.

15. GET /api/manuals?systemId={id}

Acesso: autenticado. Parâmetro: `systemId` opcional.

Fluxo: suporte recebe visão global; operador e gerente recebem manuais ativos do setor. A consulta junta o progresso do usuário autenticado e calcula `progress_percent`: páginas do PDF valem até 50% e vídeo até 50%.

16. GET /api/manuals/{id}

Acesso: autenticado com visibilidade. Fluxo: busca manual e autor; suporte pode abrir qualquer manual; os demais precisam pertencer ao mesmo setor. Sem acesso, responde 404 para não revelar a existência.

17. POST /api/manuals

Acesso: gerente de setor ou suporte. Formato: `multipart/form-data`.

Campos: título, descrição, systemId, setor, prazo, PDF, vídeo e `quiz` JSON serializado. PDF e vídeo são obrigatórios e limitados a 500 MB cada. Tipos aceitos: PDF, MP4, WebM e QuickTime.

Fluxo: gerente tem o setor forçado pelo JWT; arquivos são enviados ao Supabase ou pasta local; PDFs recebem neutralização de comandos conhecidos de impressão automática; setor é criado/reutilizado; manual é inserido; questionário é validado e salvo; responde 201.

```
curl -X POST http://localhost:4000/api/manuals \
  -H "Authorization: Bearer TOKEN" \
  -F "titulo=Cadastro" -F "systemId=1" -F "setor=Recepção" \
  -F "pdf=@manual.pdf" -F "video=@video.mp4"
```

18. PUT /api/manuals/{id}

Acesso: suporte. Formato: `multipart/form-data`.

Fluxo: aceita metadados, prazo, situação e arquivos opcionais. Ao receber novo PDF ou vídeo, copia a versão ativa para `manual_version_history`, arquiva progressos em `progress_version_history`, limpa o progresso ativo e incrementa a versão. Depois atualiza e devolve o manual.

19. DELETE /api/manuals/{id}

Acesso: suporte. Fluxo: exclui o manual. Cascatas removem progresso, perguntas, tentativas, permissões e históricos. Responde 204.

20. GET /api/progress/{manualId}

Acesso: usuário com visibilidade do manual. Fluxo: valida acesso; consulta a chave usuário/manual; se ainda não existir, devolve um objeto inicial com páginas e vídeo zerados.

21. PUT /api/progress/{manualId}

Acesso: usuário com visibilidade. Entrada: página, total de páginas, percentual do vídeo e segundos.

Fluxo: normaliza limites; executa UPSERT; une páginas sem duplicar; mantém o maior vídeo; acumula até 60 segundos por chamada; atualiza último acesso; testa PDF completo, vídeo a 90% e quiz aprovado; devolve o progresso consolidado.

```
{"page":4,"pageCount":10,"videoProgress":35.5,"seconds":30}
```

22. GET /api/progress/report

Acesso: gerente de setor ou suporte. Fluxo: cruza todos os operadores comuns com todos os manuais do setor, mesmo sem progresso. Gerente recebe apenas seu setor; suporte recebe todos. Retorna páginas lidas, vídeo, datas, prazo e versão.

23. GET /api/quiz/{manualId}

Acesso: usuário com visibilidade. Fluxo: busca perguntas por posição e remove `correct_option` da resposta. Manual sem perguntas retorna array vazio.

24. POST /api/quiz/{manualId}/attempts

Acesso: usuário com visibilidade. Entrada: objeto `answers`, com ID da pergunta como chave e índice da alternativa como valor.

Fluxo: carrega respostas corretas apenas no backend; calcula nota; aprova a partir de 70%; grava a tentativa completa; se aprovado, reavalia o progresso para permitir conclusão.

```
{"answers":{"12":0,"13":2}}
```

25. `GET /uploads/{pasta}/{arquivo}`

Acesso: público quando o armazenamento local é usado. Fluxo: middleware estático do Express serve os arquivos de `backend/uploads`. Em produção, avalie autenticação ou URLs assinadas se o conteúdo for confidencial.

Tratamento de erros

Rotas desconhecidas respondem 404. Erros assíncronos chegam ao middleware final. Falhas conhecidas de banco recebem a mensagem de indisponibilidade; outras usam a mensagem da exceção ou `Erro interno do servidor`.

Código	Significado
200	Consulta/alteração concluída
201	Recurso criado
204	Sucesso sem corpo
400	Entrada ou regra inválida
401	Token ou credencial inválida
403	Perfil sem permissão
404	Recurso invisível/inexistente
429	Login temporariamente bloqueado
500	Erro interno ou infraestrutura

Modo demonstração

Quando `DEV_DEMO_MODE=true` e o PostgreSQL está indisponível, controllers compatíveis usam `demoStore`. Os dados ficam apenas em memória e desaparecem quando o backend reinicia. Este modo não deve ser usado em produção.